# LAB MANUAL 1

## Introduction to the C# Environment and Basic Data Organization

**DHA Suffa University · Department of Computer Science**
**Data Structures Lab (C#) | CS-2007L | 2026**

| Item | Detail |
|---|---|
| Lab / Week | Lab 1 / Week 1 |
| Credits | 1(0-3) |
| Prerequisite | Programming Fundamentals |
| Duration | 3 hours |
| Assessment | Lab Task 1 (10 marks). Feeds Assignment 1 (Labs 1–4). Examinable in Quiz 1 (Labs 1–3). |
| Textbook | McMillan — *Data Structures and Algorithms Using C#* (intro chapters) |

Student Name / Reg. No.: _____ / _____

---

## 1. CLOs addressed

| CLO | Focus in this lab |
|---|---|
| CLO-1 | Environment, terminology, complexity intuition, viva |
| CLO-2 | Examples 1–6; Tasks 2, 4, 5, 6 |
| CLO-3 | XML comments, AI disclosure (Task 6, §8) |

---

## 2. Objectives

1. Set up .NET 8 + IDE; create, build, run, debug a console project.
2. Learn data / field / record / file; primitive vs non-primitive.
3. Perform six array operations: traverse, search, insert, delete, sort, merge.
4. See time–space tradeoff with measured timings.

### Learning outcomes

| LO | After this lab you can… |
|---|---|
| LO1 | Explain compiler, CLR, project structure |
| LO2 | Create/build/run via CLI and IDE |
| LO3 | Use breakpoints and Locals to fix logic errors |
| LO4 | Distinguish data/field/record/file and value vs reference |
| LO5 | Implement the six operations on an array |

| | |
|---|---|
| LO6 | Explain time–space tradeoff with Stopwatch evidence |
| LO7 | Write XML comments and an honest AI disclosure |

---

## 3. Prerequisites

- Variables, I/O, `if` / `switch`, `for` / `while`
- Basic terminal ( `cd` , list files)
- Install tools in §5 **before** the lab if possible

---

## 4. Theory (study before / during demos)

### 4.1 Pipeline

`Program.cs` → compiler → `.dll` (IL) → CLR + JIT → machine code → output

| Command | Purpose |
|---|---|
| `dotnet --version` | SDK version (need 8.0.x) |
| `dotnet new console -n Lab01 --use-program-main` | Console + explicit `Main` |
| `dotnet build` / `dotnet run` | Compile / run |
| `bin/`, `obj/` | Generated — never submit |

### 4.2 Terminology

| Term | Meaning | Example |
|---|---|---|
| Data | Raw fact | `"Ayesha"`, `3.45` |
| Field | Attribute | `Name`, `RegNo`, `Cgpa` |
| Record | One entity | one student |
| File | Records on disk | `students.txt` |

### 4.3 Value vs reference

- Value types ( `int` , …): assignment copies the value.
- Reference types (arrays, classes): assignment copies the reference.
- `Clone()` → independent array. `string` is immutable.

### 4.4 Operations & typical array costs

| Op | Cost (typical) |
|---|---|
| Traverse / linear search | O(n) |
| Insert / delete | O(n) |
| Sort (good) | O(n log n) |

| | |
|---|---|
| Merge (sorted) | O(n + m) |
| Index access | O(1) |

### 4.5 Time–space tradeoff

More memory (e.g. `HashSet`) can buy faster lookups; less memory may mean more time. Lab 2 formalizes Big-O.

### 4.6 Debugger

F5 / F9 / F10 / F11 · Locals/Watch · Focus on **logic** errors.

---

## 5. Tools

- .NET 8.0 SDK — https://dot.net
- VS 2022 (Community) **or** VS Code + C# Dev Kit
- Snipping Tool for screenshots

Verify: `dotnet --version` and `dotnet --list-sdks`

---

## 6. Solved examples (namespace `DsuLab01`)

Type each example; compare expected output; answer Observe notes.

| Ex | File(s) | LO |
|---|---|---|
| 1 | `Program.cs` Hello | LO1–2 |
| 2 | `DebugDemo.cs` off-by-one sum | LO3 |
| 3 | `TypesDemo.cs` value/ref | LO4 |
| 4 | `StudentRecord.cs` + `RecordFileDemo.cs` | LO4 |
| 5 | `ArrayOperations.cs` + demo | LO5 |
| 6 | `TradeoffDemo.cs` linear vs HashSet | LO6 |

### Example 1 — Hello

Print greeting + `Environment.Version`. Use XML comments on class/`Main`.

### Example 2 — Debug

`SumToN`: loop `i < n` → wrong sum. Fix to `i <= n` using debugger.

### Example 3 — Types

Show int copy vs array shared reference vs `Clone()` vs string immutability vs overflow wrap.

### Example 4 — Record + file

`StudentRecord` with `ToCsv` / `FromCsv`; write/read `students.txt`.

### Example 5 — Six ops

Implement Traverse, Search, Insert, Delete, Sort ( `Array.Sort` ), Merge (two **sorted** arrays). Insert/Delete return **new** arrays.

### Example 6 — Tradeoff

20,000 even numbers; 20,000 membership queries; time linear vs `HashSet` . Run **Release**:
`dotnet run -c Release`

> *Full reference source for Examples 1–6 matches the official Lab Manual 1 (CS-2007L). Keep method signatures and expected outputs identical for grading consistency.*

---

## 7. Graded tasks

Complete **Tasks 1–4** in lab. **5–6** if time, else homework.

### Task 1 — Environment report (Easy)

Print: `Environment.Version` , OS description, `ProcessorCount` , `DateTime.Today` .
Screenshots: program output + `dotnet --list-sdks` .
Written (2 lines each): (a) SDK vs runtime (b) what `dotnet build` produces and where.

### Task 2 — Student profile card (Easy)

Read name, reg no, programme, CGPA with `double.TryParse` retry.
Print card; scholarship if CGPA ≥ 3.00.

### Task 3 — Types table + classification (Easy–Medium)

**A:** Table of `sizeof` / Min / Max for byte, short, int, long, float, double; also char, bool.
**B:** Classify: (i) `"Karachi"` (ii) column Age (iii) one attendance row (iv) `attendance.csv` (v) `3.14`
**C:** `int[] y = x; y[0]=9;` — values of `x[0]` , `y[0]` using words *reference* and *heap*.

### Task 4 — BuggyStats (Medium)

Three deliberate bugs. Tests: `{4,5,6,7}` → avg 5.5, max 7; `{-7,-3,-9}` → max -3.

```
// Bugs: (1) i <= Length  (2) int division  (3) max starts at 0
public static double Average(int[] values) { /* ... */ }
public static int Maximum(int[] values) { /* ... */ }
```

For each bug: symptom, root cause, corrected line, syntax/runtime/logic.
One debugger screenshot per bug. Submit fixed class + both outputs.

### Task 5 — Enrolment manager (Medium–Hard)

Arrays only (no `List<T>` ). Menu: Display, Search, Enrol@position, Remove by ID, Sort, Merge, Exit.
Validate input; reject duplicate IDs; XML comments; reuse Example 5 style ops.

### Task 6 — Sensor duplicates (Challenging)

`HasDuplicateNested` vs `HasDuplicateHashSet`.
n = 5k, 10k, 20k, 40k; no-duplicate data; Release; 3 runs; median table; 6–8 line conclusion (cloud vs edge); AI disclosure.

---

## 8. Post-lab (Assignment 1 contribution)

**Q1 (CLO-1):** Short answers — pipeline; value vs ref; data/field/record/file (library catalogue); six ops + costs; one tradeoff example.

**Q2 (CLO-2):** `Book` class like `StudentRecord`; five books → `books.txt`; search by ISBN.

**Q3 (CLO-2/C4):** Instrument `Search` with comparison count; n=1000 best/middle/worst; relate to n.

**Reflection:** hardest Task 4 bug; when save memory; one readability habit.

**AI disclosure template:** tool / purpose / changed / verified / can explain Yes-No + signature.

### Submission

IDE build clean · screenshots · PDF to LMS · no `bin` / `obj`.

---

## 9. Rubric (Lab Task /10)

| Component | CLO | Marks |
|---|---|---|
| Demonstration (run + debug) | CLO-2 | 2 |
| Task completion | CLO-2 | 3 |
| Code quality + responsible practice | CLO-3 | 2 |
| Viva / concepts | CLO-1 | 3 |
| **Total** | | **10** |

---

## 10. Viva hotlist

See `slides/C_viva-bank.md` (compiler vs CLR, value/ref, insert O(n), debugger, HashSet cost, AI disclosure).